Complexity of Algorithm

Lecture 03

Analyzing Algorithms

Analyze algorithms to gauge:

- Time complexity (running time)
- Space complexity (memory use)

Input size is indicated by a number *n*

- sometimes have multiple inputs, *e.g.* m and n

Running time is a function of *n*

n, n^2, $n \log n$, $18 + 3n(\log n^2) + 5n^3$

RAM Model

RAM (random access machine)

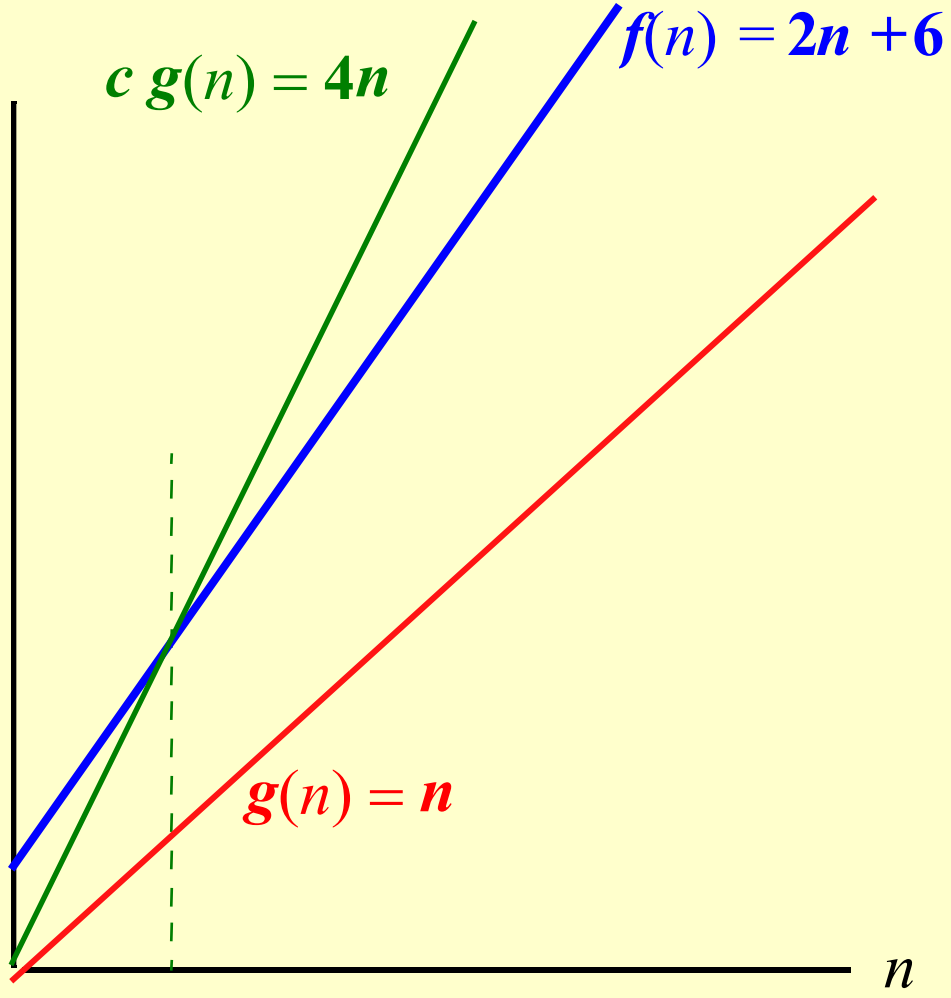
- Ideal single-processor machine (serialized operations)
- “Standard” instruction set (load, add, store, etc.)
- All operations take 1 time unit (including, for our purposes, each C++ statement)

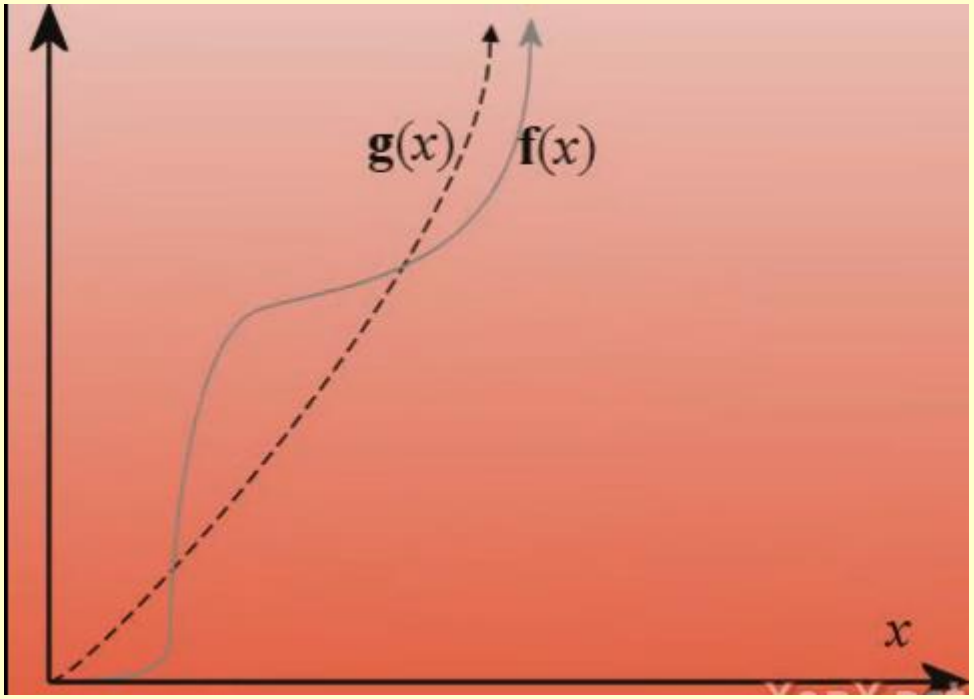
Order Notation (aka Big-O)

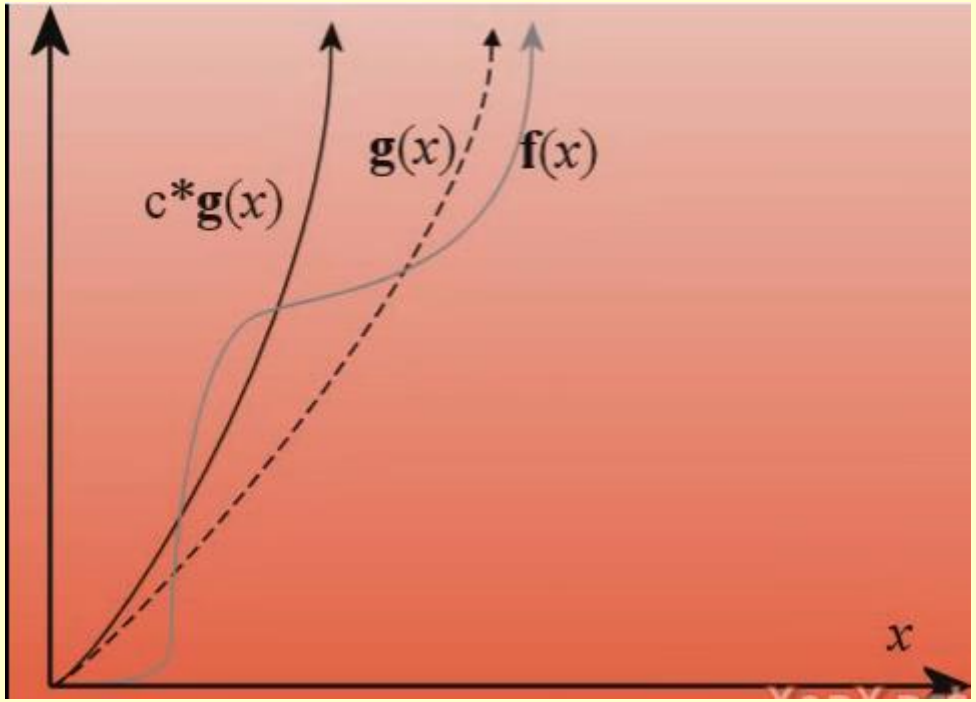
Big-O	$T(n) = O(f(n))$ Exist positive constants c, n_0 such that $T(n) \leq cf(n)$ for all $n \geq n_0$	Upper bound
Omega	$T(n) = \Omega(f(n))$ Exist positive constants c, n_0 such that $T(n) \geq cf(n)$ for all $n \geq n_0$	Lower bound
Theta	$T(n) = \theta(f(n))$ $T(n) = O(f(n))$ AND $T(n) = \Omega(f(n))$	Tight bound
little-o	$T(n) = o(f(n))$ $T(n) = O(f(n))$ AND $T(n) \mathrel{!=} \theta(f(n))$	Strict upper bound

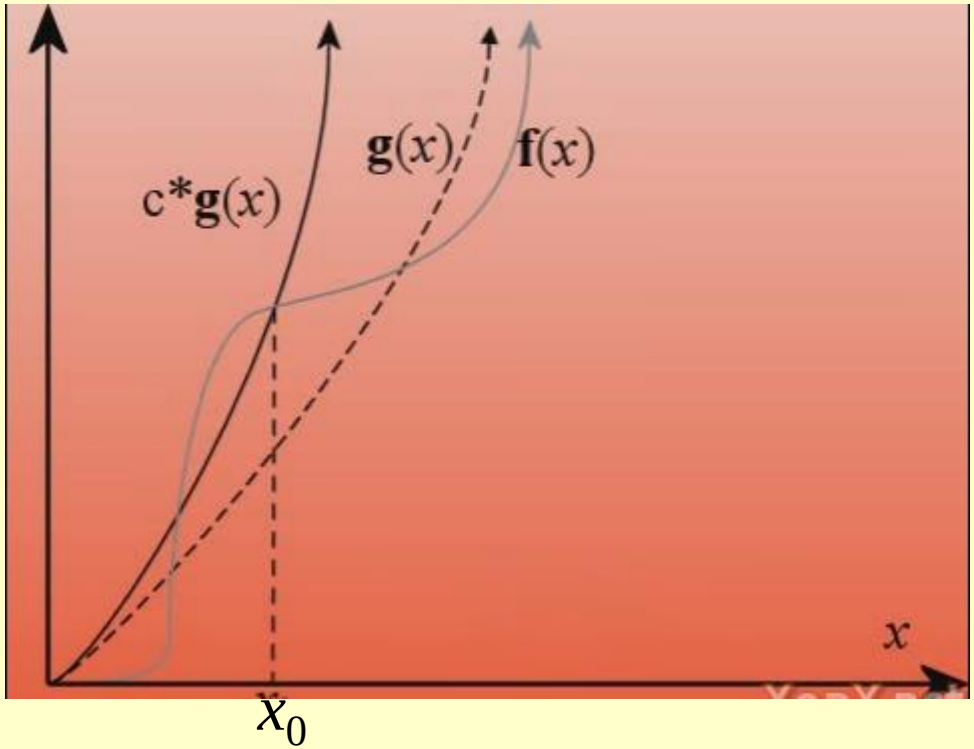
Graphic Illustration

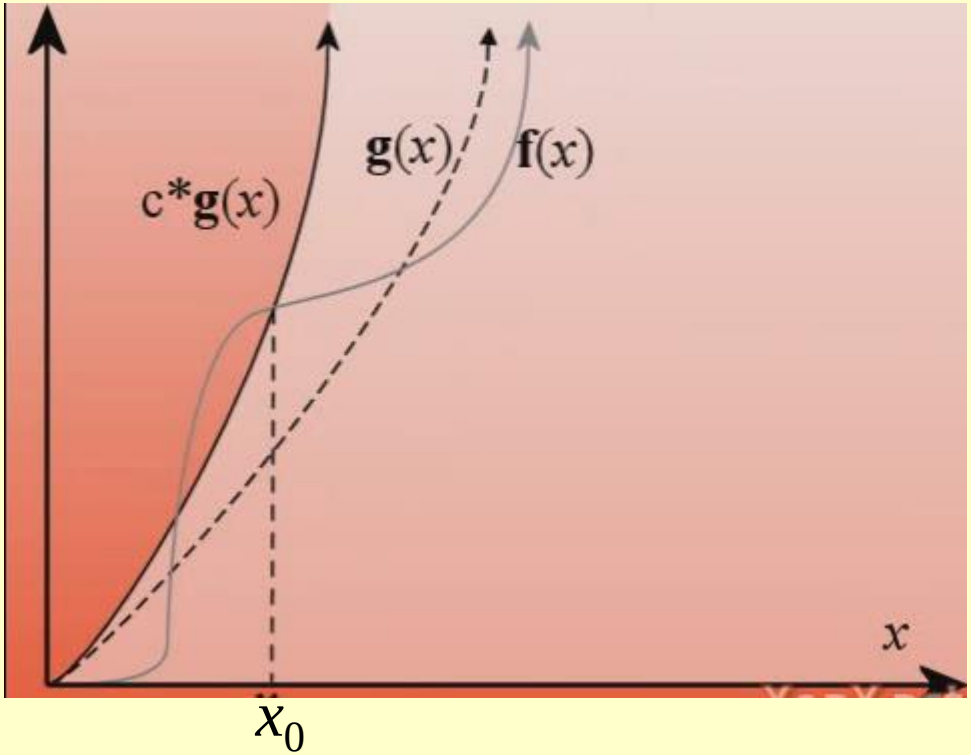
- f(n) = 2n+6
- Conf. def:
  - Need to find a function g(n) and a const. c and a constant n0 such as f(n) < cg(n) when n>n0
- g(n) = n and c = 4 and n0=3
- → f(n) is O(n)
- The order of f(n) is n




g(x)
f(x)
x


c*g(x)
g(x)
f(x)
x


c*g(x)
g(x)
f(x)
x
x_0

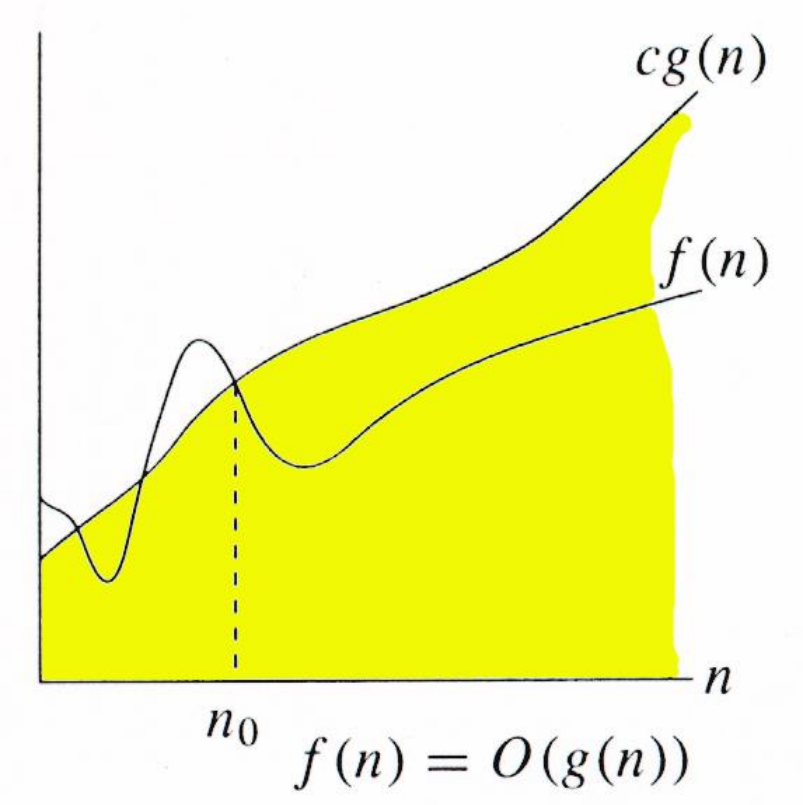

c*g(x)
g(x)
f(x)
x
x_0

O-notation

For function $g(n)$, we define $O(g(n))$, big-O of n, as the set:

$O(g(n)) = \{f(n) :$
$\exists$ **positive constants** c **and** n_0,
such that $\forall n \geq n_0$,
we have $0 \leq f(n) \leq cg(n)\}$

Intuitively: Set of all functions whose *rate of growth* is the same as or lower than that of $g(n)$.



Simplifying with Big-O

By definition, Big-O allows us to:

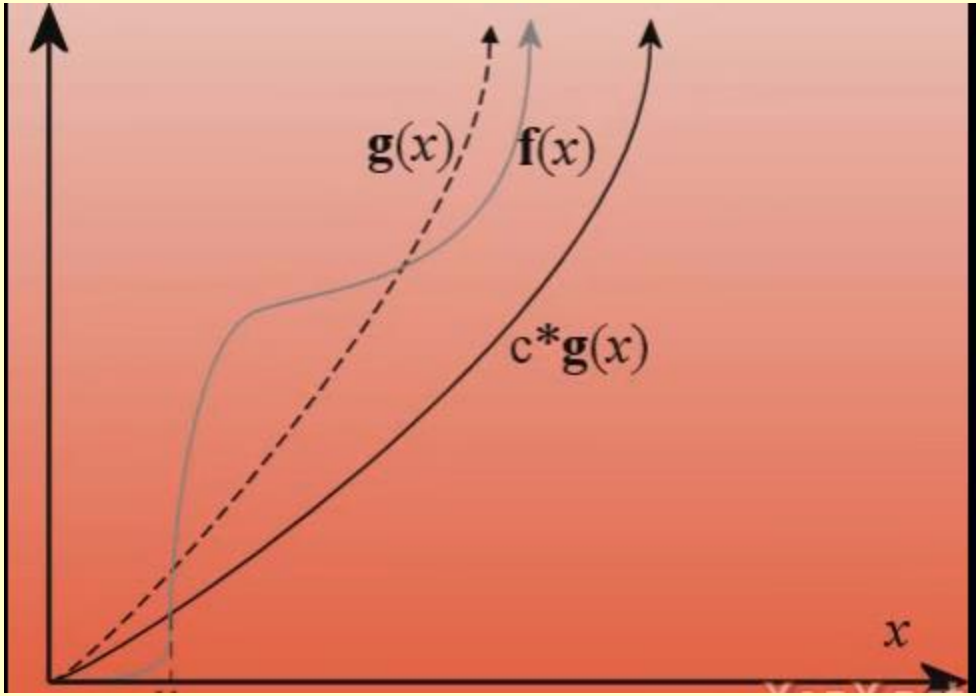
Eliminate low order terms

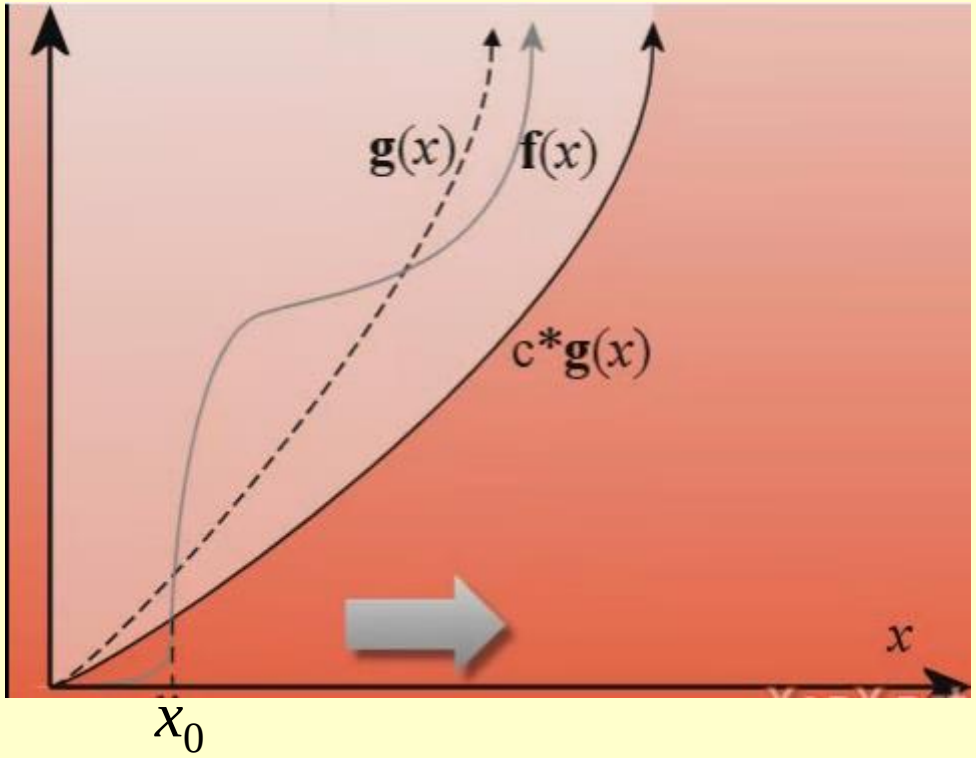
- $4n + 5 \Rightarrow 4n$
- $0.5\, n \log n - 2n + 7 \Rightarrow 0.5\, n \log n$

Eliminate constant coefficients

- $4n \Rightarrow n$
- $0.5\, n \log n \Rightarrow n \log n$
- $\log n^2 = 2 \log n \Rightarrow \log n$
- $\log_3 n = (\log_3 2) \log n \Rightarrow \log n$

But when might constants or low-order terms matter?


g(x)
f(x)
c*g(x)
x

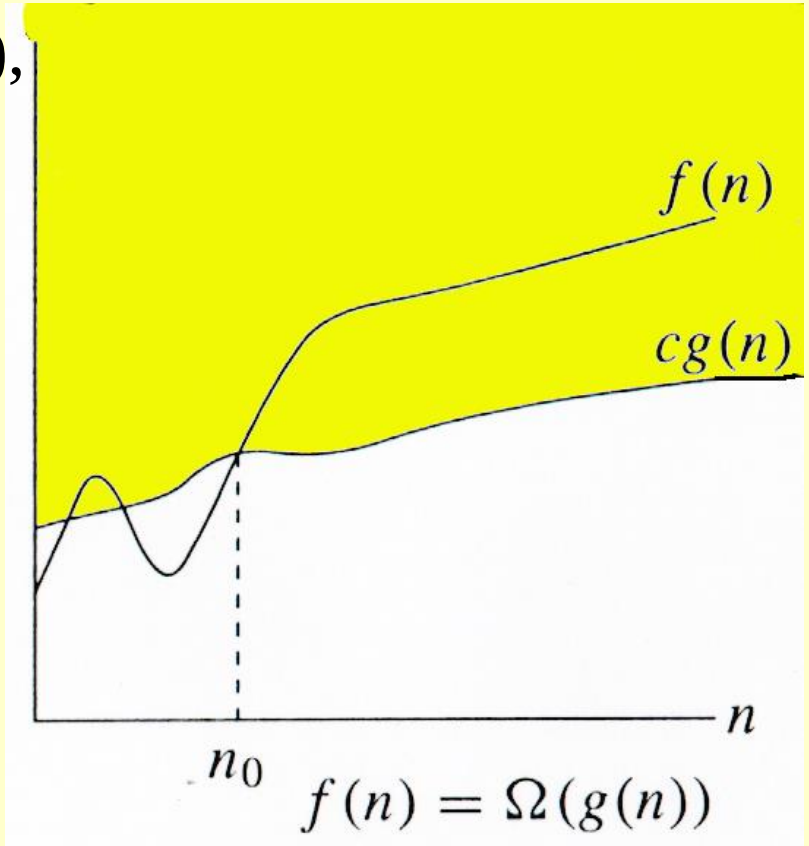

g(x)
f(x)
c*g(x)
x
x0

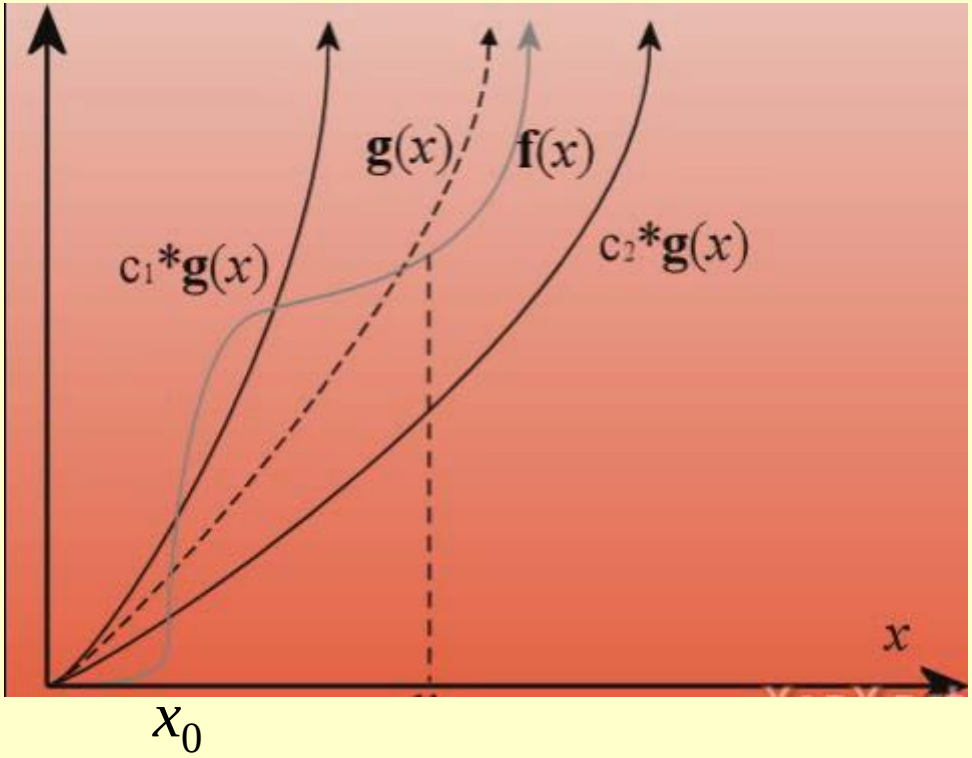
Ω -notation

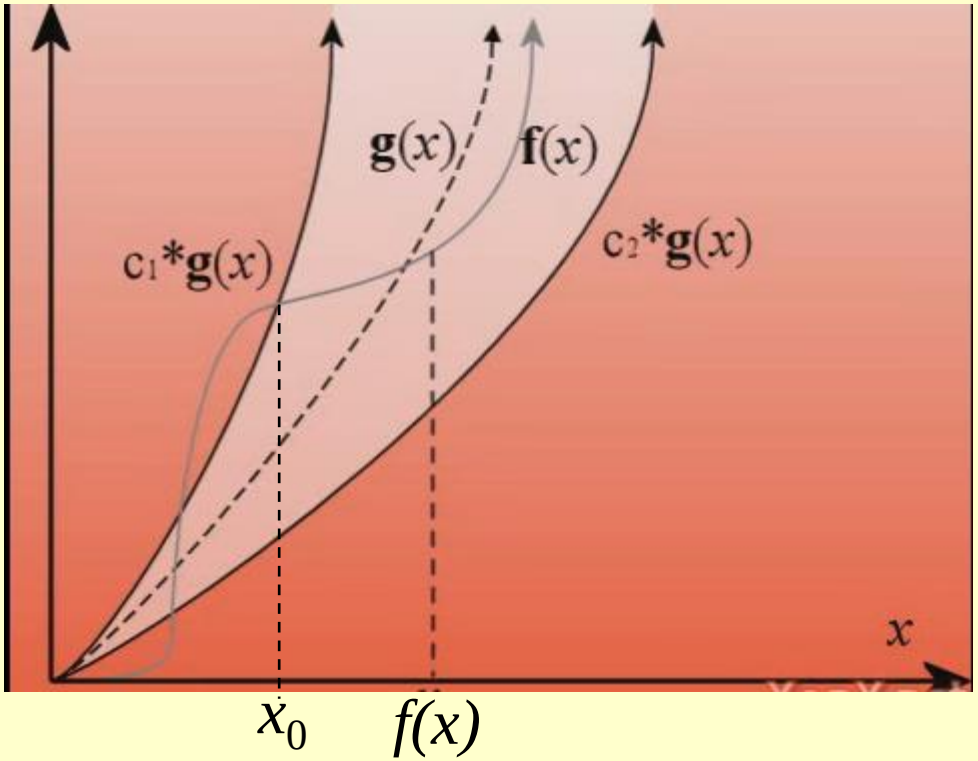
For function $g(n)$, we define $\Omega(g(n))$, big-Omega of n, as the set:

$\Omega(g(n)) = \{f(n) :$
$\exists$ positive constants c and n_0,
such that $\forall n \geq n_0$,
we have $0 \leq cg(n) \leq f(n)\}$

Intuitively: Set of all functions f(n) whose *rate of growth* is the same as or higher than that of $g(n)$.



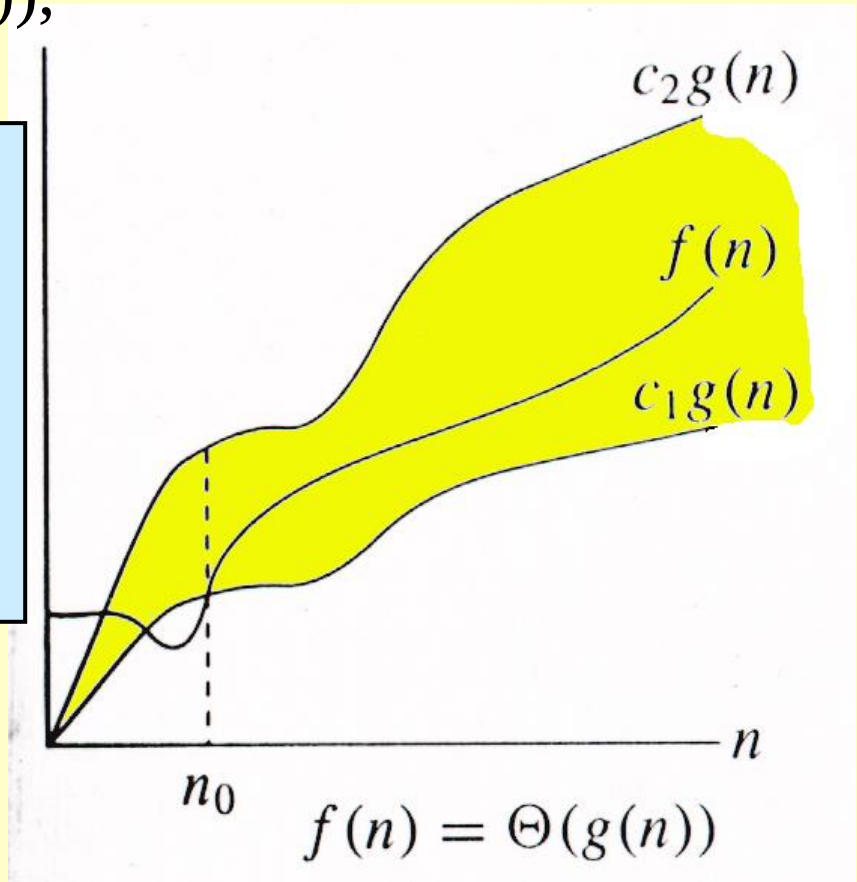

g(x)
f(x)
c1*g(x)
c2*g(x)
x
x0


g(x)
f(x)
c1*g(x)
c2*g(x)
x
x0
f(x)

Θ-notation

For function $g(n)$, we define $\Theta(g(n))$, big-Theta of n, as the set:

$\Theta(g(n)) = \{f(n) :$
$\exists$ **positive constants** c_1, c_2, **and** n_0,
such that $\forall n \geq n_0$,
we have $0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$
$\}$



Intuitively: Set of all functions that have the same *rate of growth* as $g(n)$.

$g(n)$ is an *asymptotically tight bound* for $f(n)$.

Example

$f(n)$ is $O(g(n))$
if C and n_0 gives
$f(n) <= Cg(n)$
for $\forall n \geq n_0$

$f(n) = 4n^2+16n+2$
Is $f(n) => O(n^4)$?

n	**$4n^2+16n+2 < n^4$**	
0	$2 \not< 0$	C=1
1	$22 \not< 1$	
2	$50 \not< 16$	
3	$86 \not< 81$	
4	$150 < 256$	$n_0 = 4$

$f(n) < g(n^4)$ for $n>=4$ with C=1
So $f(n) => O(n^4)$ exists

Example

$f(n)$ is $\mathbf{O}(g(n))$
if C and n_0 gives
$f(n) <= Cg(n)$
for all $n > n_0$

$f(n) = 4n^2+16n+2$
Is $f(n) => \mathbf{O}(n^4)$?

n	$4n^2+16n+2 < Cn^4$	
0	$2 \not< 0$	C=1
1	$22 \not< 1$	
2	$50 \not< 16$	
3	$86 \not< 81$	
4	$150 < 256$	$n_0 = 4$

$f(n)$ is $\mathbf{\Omega}(g(n))$
if C and n_0 gives
$f(n) >= Cg(n)$
for all $n < n_0$

$f(n) = 4n^2+16n+2$
Is $f(n) => \mathbf{\Omega}(n^2)$?

$\mathbf{4n^2+16n+2 > Cn^2=n^2}$

Always true because of constant

$f(n) > g(n^2)$ for n=0 with C=1
So $f(n) => \mathbf{\Omega}(n^2)$ exists

Example

$f(n)$ is $\mathbf{O}(g(n))$
if C and n_0 gives
$f(n) <= Cg(n)$
for $\forall n \geq n_0$

$f(n) = 4n^2+16n+2$
Is $f(n) => \mathbf{O}(n^4)$?

n	$4n^2+16n+2 < Cn^4$	
0	$2 \not< 0$	C=1
1	$22 \not< 1$	
2	$50 \not< 16$	
3	$86 \not< 81$	
4	$150 < 256$	$n_0 = 4$

$f(n)$ is $\mathbf{\Omega}(g(n))$
if C and n_0 gives
$f(n) >= Cg(n)$
for $\forall n \geq n_0$

$f(n) = 4n^2+16n+2$
Is $f(n) => \mathbf{\Omega}(n^3)$?

n	$4n^2+16n+2 > Cn^3=Cn(n^2)$	
0	$2 > 0$	C=1
1	$22 > 1$	
2	$50 > 8$	
3	$86 < 27$	
7	$310 < 343$	$n_0 = 7$

$f(n) < g(n^3)$ for $n>7$ with C=1
So $f(n) => \mathbf{\Omega}(n^3)$ not exists

Example

$f(n)$ is $\Theta(g(n))$ IFF
i) $f(n)$ is $\mathbf{O}(g(n))$
ii) $f(n)$ is $\mathbf{\Omega}(g(n))$

$f(n) = 4n^2+16n+2$
Is $f(n) \Rightarrow \Theta(n^2)$?

Need to check
Is $f(n) \Rightarrow O(n^2)$?

$f(n) > g(n^2)$ for $n>=17$ with $C=5$
So $f(n) \Rightarrow \mathbf{O}(n^2)$ exists

n	$4n^2+16n+2 < Cn^2$	
0	$2 \not< 0$	C=5
1	$22 \not< 5$	
2	$50 \not< 20$	
3	$86 \not< 45$	
17	$1430 < 1445$	

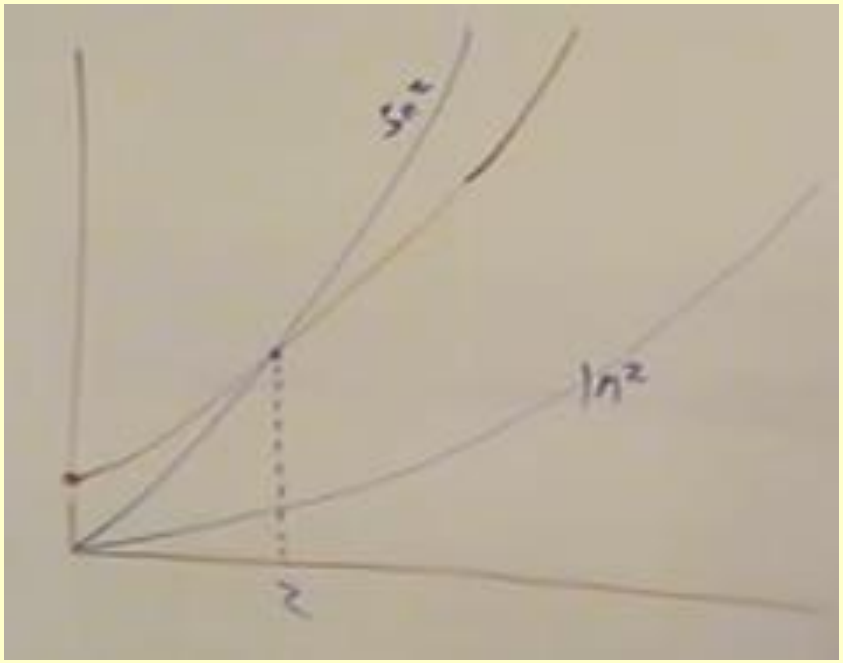
$\mathbf{4n^2+16n+2 > Cn^2=n^2}$
Always true because of constant

$f(n) > g(n^2)$ for $n_0=0$ with $C=1$
So $f(n) \Rightarrow \mathbf{\Omega}(n^2)$ exists

$f(n) \Rightarrow \Theta(n^2)$ exists

Importance BIG O in Programming

Let T(n) is the time taken to finish execution



n	$4n^2$	16n	2
1	4	16	2
2	16	32	2
3	36	48	2
4	64	64	2
5	100	80	2
6	144	96	2

$T(n) = 4n^2+16n+2$

$T(n) \Rightarrow \Theta(n^2)$ means the term n^2 dominates when n is large

Classes of Functions

- Let $\{\boldsymbol{g}(\boldsymbol{n})\}$ denote the class (set) of functions that are $\boldsymbol{O}(\boldsymbol{g}(\boldsymbol{n}))$
- We have
  $\{\boldsymbol{n}\} \subset \{\boldsymbol{n}^2\} \subset \{\boldsymbol{n}^3\} \subset \{\boldsymbol{n}^4\} \subset \{\boldsymbol{n}^5\} \subset \ldots$
  where the containment is strict

Big-Oh Rules

- If is $\boldsymbol{f}(\boldsymbol{n})$ a polynomial of degree $\boldsymbol{d}$, then $\boldsymbol{f}(\boldsymbol{n})$ is $\boldsymbol{O}(\boldsymbol{n}^{\boldsymbol{d}})$, i.e.,
  1. Drop lower-order terms
  2. Drop constant factors
- Use the smallest possible class of functions
  - Say "$2\boldsymbol{n}$ is $\boldsymbol{O}(\boldsymbol{n})$" instead of "$2\boldsymbol{n}$ is $\boldsymbol{O}(\boldsymbol{n}^2)$"
- Use the simplest expression of the class
  - Say "$3\boldsymbol{n} + 5$ is $\boldsymbol{O}(\boldsymbol{n})$" instead of "$3\boldsymbol{n} + 5$ is $\boldsymbol{O}(3\boldsymbol{n})$"

Asymptotic analysis - terminology

- Special classes of algorithms:

  logarithmic: **$O(\log n)$**

  linear: **$O(n)$**

  quadratic: **$O(n^2)$**

  polynomial: **$O(n^k)$, $k \geq 1$**

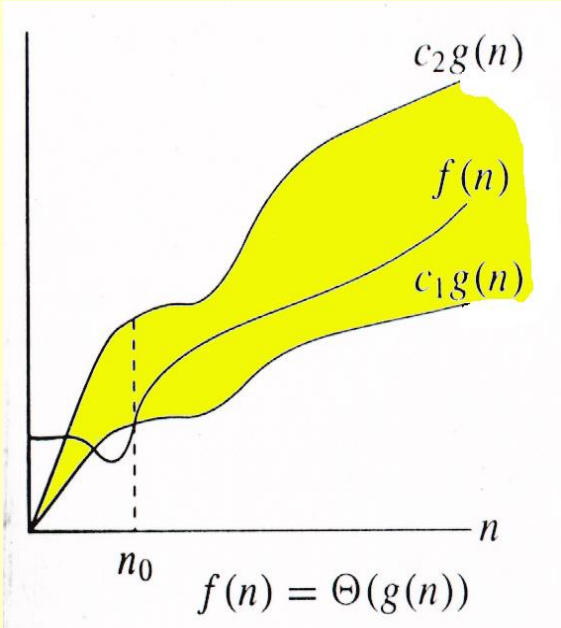
  exponential: **$O(a^n)$, $n > 1$**

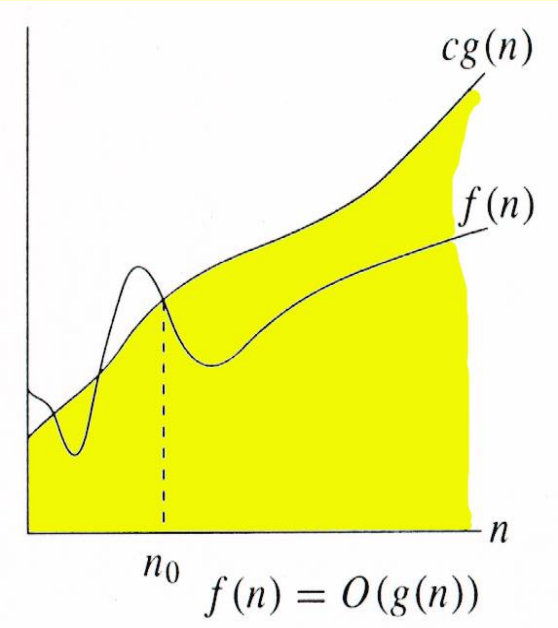
- Polynomial vs. exponential ?
- Logarithmic vs. polynomial ?

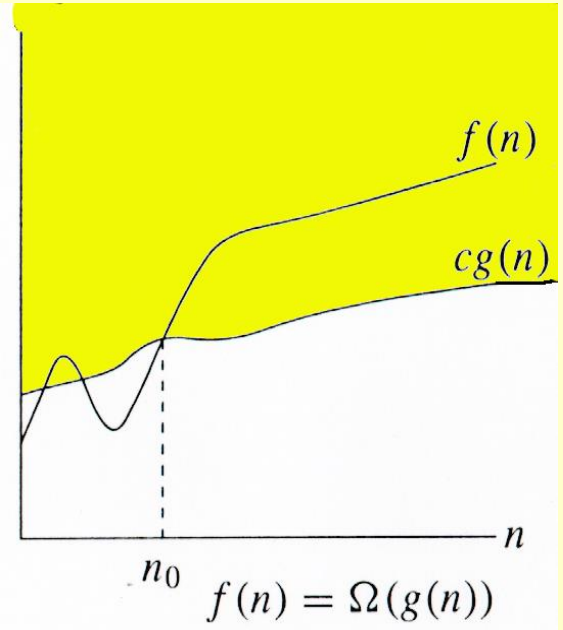
Some Numbers

log n	n	n log n	n^2	n^3	2^n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4096	65536
5	32	160	1024	32768	4294967296

Relations Between Θ, *O*, Ω







Relations Between Θ, Ω, O

Theorem : For any two functions $g(n)$ and $f(n)$,

$$f(n) = \Theta(g(n)) \text{ iff}$$

$$f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n)).$$

- I.e., $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$
- In practice, asymptotically tight bounds are obtained from asymptotic upper and lower bounds.

Review on Summations

```
MaxSubvector(A, n)
        maxsum ← 0;
        for i ← 1 to n
            do for j = i to n
                    sum ← 0
                    for k ← i to j
                        do sum += A[k]
                    maxsum ← max(sum, maxsum)
        return maxsum
```

- $T(n) = \sum_{i=1}^{n} \sum_{j=i}^{n} \sum_{k=i}^{j} 1$
- NOTE: This is not a simplified solution. What *is* the final answer?

Nested Loops

```
for i = 1 to n do
  for j = 1 to n do
    sum = sum + 1
```

$$\sum_{i=1}^{n} \sum_{j=1}^{n} 1 = \sum_{i=1}^{n} n = n^2$$

Dependent Nested Loops

```
for i = 1 to n do
  for j = i to n do
    sum = sum + 1
```

$$\sum_{i=1}^{n} \sum_{j=i}^{n} 1 = \sum_{i=1}^{n} (n-i+1) = \sum_{i=1}^{n} (n+1) - \sum_{i=1}^{n} i =$$

$$n(n+1) - \frac{n(n+1)}{2} = \frac{n(n+1)}{2} \approx n^2$$

Review on Summations

- **Constant Series:** For integers a and b, $a \le b$,

$$\sum_{i=a}^{b} 1 = b - a + 1$$

- **Linear Series (Arithmetic Series):** For $n \ge 0$,

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

- **Quadratic Series:** For $n \ge 0$,

$$\sum_{i=1}^{n} i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Review on Summations

- **Cubic Series:** For $n \geq 0$,

$$\sum_{i=1}^{n} i^3 = 1^3 + 2^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

- **Geometric Series:** For real $x \neq 1$,

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1}$$

For $|x| < 1$, $$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$$

Review on Summations

- **Linear-Geometric Series:** For $n \geq 0$, real $c \neq 1$,

$$\sum_{i=1}^{n} ic^i = c + 2c^2 + \cdots + nc^n = \frac{-(n+1)c^{n+1} + nc^{n+2} + c}{(c-1)^2}$$

- **Harmonic Series:** nth harmonic number, $n \in \mathrm{I}^+$,

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

$$= \sum_{k=1}^{n} \frac{1}{k} = \ln(n) + O(1)$$

Review on Summations

- **Telescoping Series:**

$$\sum_{k=1}^{n} a_k - a_{k-1} = a_n - a_0$$

- **Differentiating Series:** For $|x| < 1$,

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}$$